Graph Data Structure and Algorithms in Python

All code available on GitHub, within the Zip File

Task 1 - Implement and Extend Graph Basics

```
-- Undirected Graph --
A: ['B']
B: ['A', 'C']
C: ['B']

-- Directed Graph --
A: ['B']
B: ['C']
C: []

-- Weighted Directed Graph --
A: [('B', 15)]
B: [('C', 34)]
C: []

-- Removal Test --
Initial graph:
A: ['B']
B: ['A', 'C']
C: ['B']
After removing edge A-B:
A: []
B: ['C']
C: ['B']
After removing vertex C:
A: []
B: []
```

Task 2: Breadth First Search (BFS) Implementation

```
============================================================
TASK 2: BFS Traversal
============================================================
A: ['B', 'C']
B: ['A', 'D']
C: ['A', 'D']
D: ['B', 'C']

BFS starting from vertex 'A':
Visited vertices in BFS order: ['A', 'B', 'C', 'D']
```

Task 3: Depth First Search (DFS) Implementation

```
============================================================
TASK 3: DFS Traversal
============================================================

DFS starting from vertex 'A':
Visited vertices in DFS order: ['A', 'B', 'D', 'C']
```

Task 4: Cycle Detection in Undirected Graphs

```
============================================================
TASK 4: Cycle Detection (Undirected)
============================================================
Does cycle_graph have a cycle?   True
Does acyclic_graph have a cycle? False
```

Task 5: Experiment with Weighted and Directed Graphs

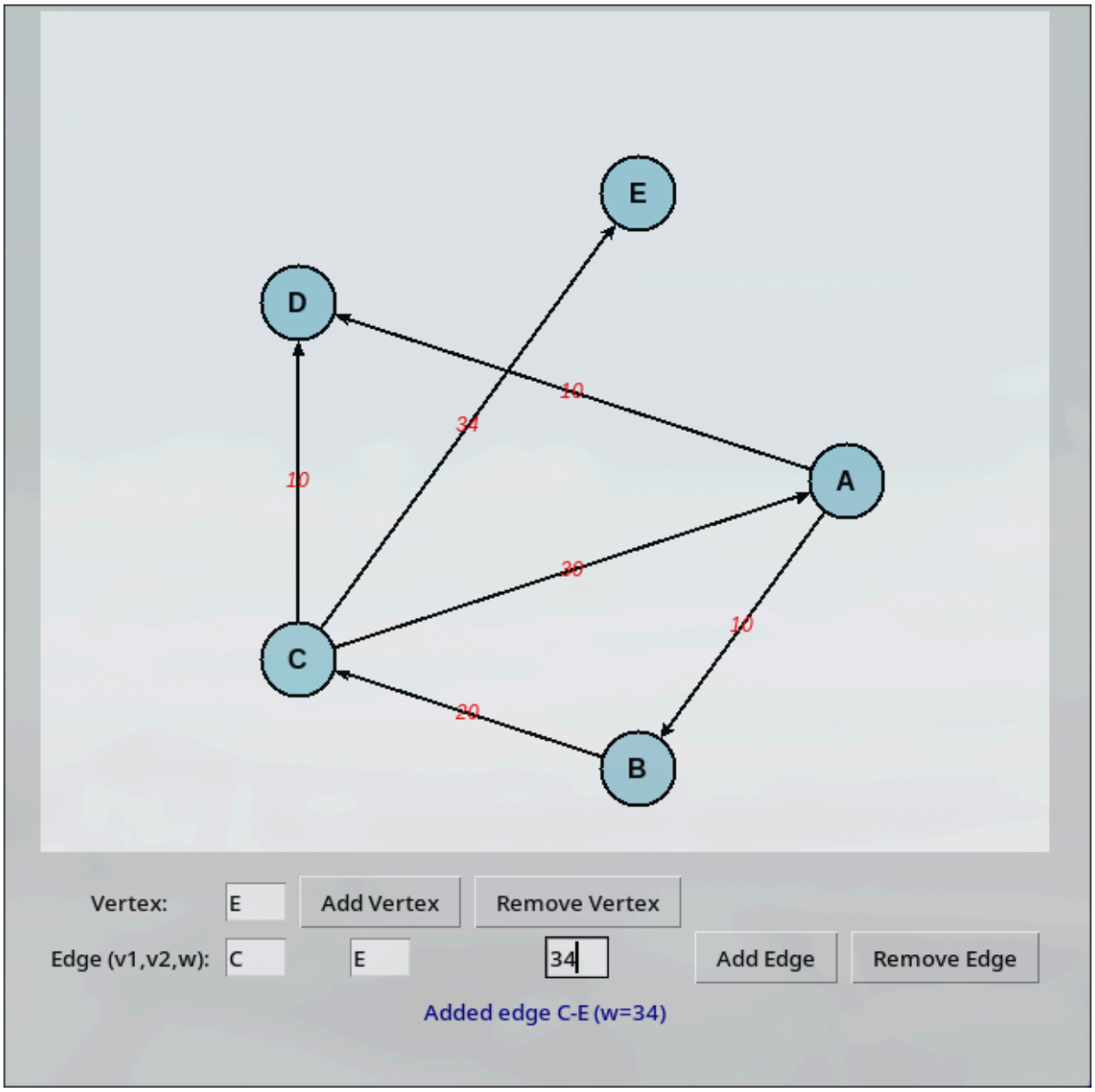
```
============================================================
TASK 5: Weighted Directed Graph
============================================================
Weighted Directed Graph:
A: [('B', 15), ('C', 50)]
B: [('C', 34)]
C: []

Weights dict: {('A', 'B'): 15, ('B', 'C'): 34, ('A', 'C'): 50}
```

In an unweighted graph, edges are treated as binary relationships: two vertices are either connected or they are not. Introducing weights transforms each edge into a quantitative relationship, allowing the graph to model real-world systems such as road networks (where weights represent distances or travel times), telecommunication networks (where weights might represent bandwidth or latency), or financial systems (where weights encode costs or probabilities). This additional information fundamentally changes which algorithms are appropriate and how they must be implemented.

The most immediate consequence of weights is that simple BFS can no longer reliably find the "shortest" path between two vertices. BFS assumes all edges have equal cost, so it finds the path with the fewest hops. In a weighted graph, the fewest-hop path may be far more expensive than a longer route with lower-weight edges. To find the true least-cost path, more sophisticated algorithms are required, most notably Dijkstra's algorithm, which uses a priority queue to always expand the lowest-cost known path first, and the Bellman-Ford algorithm, which can additionally handle negative edge weights at the cost of higher time complexity.

Weights also affect how the graph is stored and queried. In this implementation, weights are kept in a separate dictionary keyed by `(vertex1, vertex2)` tuples. While this keeps the adjacency list clean, it means every edge lookup now requires an additional dictionary access to retrieve the weight. For dense graphs with many edges, this overhead is negligible, but it is worth noting that alternative representations,s such as adjacency matrices, can store weights directly in the matrix cell, making retrieval O(1) at the cost of $O(V^2)$ memory. The choice of representation, therefore, depends on whether the graph is sparse (few edges relative to vertices, favouring adjacency lists) or dense (many edges, where a matrix may be more practical). Beyond pathfinding, weights influence algorithms for minimum spanning trees (Prim's and Kruskal's algorithms select edges by weight to connect all vertices at minimum total cost), network flow problems (where edge weights represent capacities), and clustering methods that group vertices by edge-weight similarity. In short, weights elevate a graph from a simple topological structure to a rich quantitative model, and every major operation must be redesigned to take that numeric information into account.

Task 6: Visualisation Graph Data Structure with Tkinter



Task 7: Benchmarking Graph Operations

```
--- Benchmarking: 10 vertices, 20 edges ---
Added 10 vertices in 0.0000 seconds
Added 20 edges in 0.0001 seconds
BFS completed in 0.0002 seconds
DFS completed in 0.0000 seconds

--- Benchmarking: 100 vertices, 300 edges ---
Added 100 vertices in 0.0001 seconds
Added 298 edges in 0.0005 seconds
BFS completed in 0.0001 seconds
DFS completed in 0.0001 seconds

--- Benchmarking: 1000 vertices, 3000 edges ---
Added 1000 vertices in 0.0004 seconds
Added 3000 edges in 0.0048 seconds
BFS completed in 0.0007 seconds
DFS completed in 0.0011 seconds
```

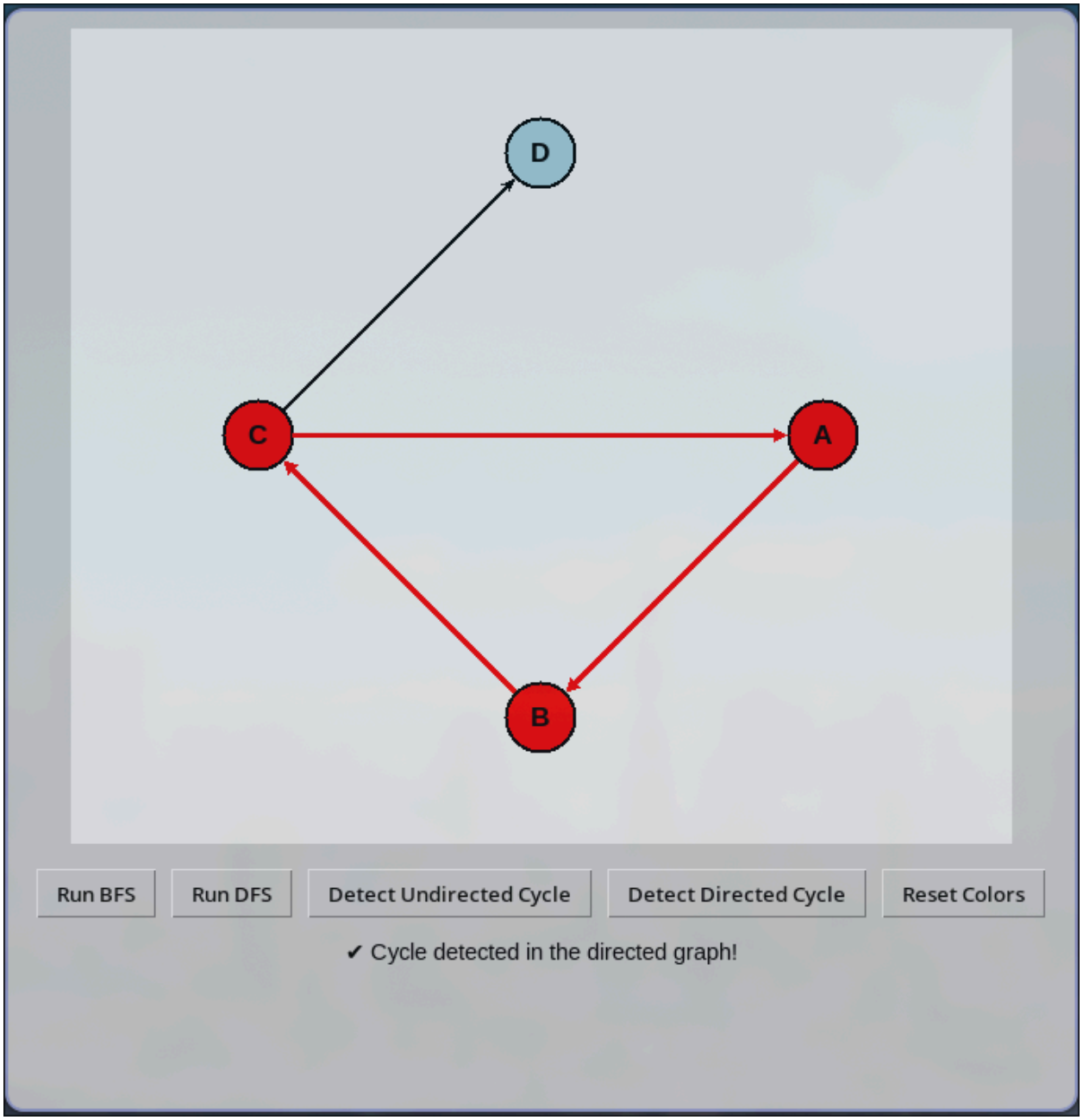
The benchmarking results demonstrate a clear and expected pattern: as the number of vertices and edges grows, the time required for every graph operation increases, though the relationship is not always linear. Adding vertices is extremely fast at all scales because it is simply a dictionary insertion, an O(1) operation. Adding edges is similarly cheap per operation, but the cumulative time grows with the number of edges being inserted, remaining roughly proportional to the edge count. For 1,000 vertices and 3,000 edges, the total time is still only a few milliseconds, confirming that adjacency-list construction scales well for sparse graphs.

The more revealing results come from BFS and DFS. Both algorithms must visit every reachable vertex and traverse every edge along the way, giving them a time complexity of O(V + E) where V is the number of vertices and E is the number of edges. In practice, this means performance is governed not just by how many vertices exist, but by how densely they are connected. A graph with 1,000 vertices but only 1,000 edges (sparse) will be traversed far more quickly than one with 1,000 vertices and 499,500 edges (a near-complete graph). This is why graph density, the ratio of actual edges to the maximum possible edges, is often a more important performance factor than vertex count alone. The benchmarking here uses a sparse graph (roughly 3 edges per vertex), which is why traversal times remain low; in a denser graph, both traversal algorithms would take meaningfully longer as each vertex expansion would require examining far more neighbours.

Structure also matters beyond density. Traversal performance degrades when graphs have very high-degree "hub" vertices, because each visit to such a vertex requires iterating over a long neighbour list. Similarly, disconnected graphs can cause algorithms to terminate early (if they

only explore from one starting vertex) or require additional outer loops to visit all components, changing the effective work done. For cycle detection, recursive DFS carries an additional concern: deeply nested recursion on large graphs can exhaust Python's call stack, triggering a `RecursionError`. This is why iterative implementations using an explicit stack are preferred in production systems for large inputs. Memory is another consideration; adjacency lists use O(V + E) space, which is efficient for sparse graphs, but even this becomes significant at millions of vertices. The benchmarking results, therefore, paint only part of the picture: choosing the right representation and algorithm for a graph's specific size, density, and connectivity pattern is essential for building systems that remain performant as data scales.

Challenge Task- Extended Graph Data Structure Visualizer



As you can see, both Directed & Undirected cycle detection now work. To switch between Directed and Undirected, the following boolean variable has to be changed

```
USE_DIRECTED = True    # change to True to test directed cycle detection.
False to test undirected
```